
TP Applications de la Cryptographie

TP à réaliser par groupe de 3

Partie 1

Introduction

OpenSSL est une boîte à outils de chiffrement comportant deux bibliothèques (*libcrypto* et *libssl*) fournissant respectivement une implémentation des algorithmes cryptographiques et du protocole de communication SSL/TLS, ainsi qu'une interface en ligne de commande (Linux avec des portages sous Windows). Cette dernière permet notamment :

1. La création de clés asymétriques (RSA, etc.)
2. La création de certificats X509
3. Des opérations cryptographiques telles que le hachage (SHA256, etc.) le chiffrement et déchiffrement (AES, etc.)
4. La réalisation de tests de clients et serveurs SSL/TLS
5. La signature et le chiffrement de courriels (protocole S/MIME)

Pour connaître toutes les fonctionnalités d'OpenSSL le plus simple est de consulter le manuel

<code>man openssl</code>	ou
<code>openssl [commande] -help</code>	ou encore
<code>man <commande></code>	

La syntaxe générale de la commande d'OpenSSL est :

<code>\$openssl commande</code>

Etude des certificats, de leurs caractéristiques

Nous allons d'abord récupérer le certificat du site web <https://cyber.gouv.fr/>

1. Quel protocole sécurise cette connexion lors de la consultation d'un navigateur ?

Pour cela utilisez la commande **s_client** d'OpenSSL

(Aide : afin d'accélérer la fin de la commande, à la fin de la ligne de commande OpenSSL rajoutez :

- Sous Windows cmd **2>nul < nul**

- Sous Windows Powershell **2>nul** (à la fin) et **Write-Host \$null |** (au début)
- Sous Linux **2>/dev/null </dev/null**

)

Récupérez le certificat (se situant entre les balises -----BEGIN CERTIFICATE----- et -----END CERTIFICATE-----, **y compris ces balises**) dans le fichier *cyber.gouv.fr.pem*

En vous aidant du résultat de la commande OpenSSL :

2. Quelle est la longueur de la chaîne de certification ?
3. Quel est le nom de l'autorité de certification **racine** ?
4. Quel est le nom de l'autorité de certification **subordonnée** ?
5. Quelle est la taille de la clé publique du site *cyber.gouv.fr.pem* ?

(Indice : OpenSSL précise des acronymes comme

s: (pour Subject) **i:** (pour Issuer) **v:** (pour Validity)

)

Plus en détail

Affichez le certificat contenu dans le fichier *cyber.gouv.fr.pem* récupéré précédemment à l'aide de la commande **x509** sous un **format lisible**

6. Observez le sujet du certificat. De quel type de certificat s'agit-il ?
7. Ce certificat peut-il être utilisé pour signer d'autre certificat ?
Argumenter la réponse
8. Ce certificat peut-il être utilisé pour signer du courrier électronique ?
Argumenter la réponse
9. A quel(s) endroit(s) trouve-t-on la CRL de l'autorité de certification ayant émis ce certificat ?
10. Observer le numéro de série. En quel format est-il ? Le convertir en décimal et émettre une observation à ce sujet.
11. Affichez la clé publique du certificat.
12. Affichez la clé privée du certificat.

Récupérez la CRL de l'autorité de certification ayant émis ce certificat (avec un navigateur ou en ligne de commande avec **wget** par exemple)

Affichez les détails de la CRL en ligne de commande à l'aide de la commande OpenSSL **crl** sous un **format lisible en limitant la sortie de la commande à 50 lignes** (« | head -n50 » sous Linux ou « | Select -First 20 » en Powershell sous Windows)

13. Combien de temps est-elle valide ?
14. Les raisons de révocation des 1ers certificats révoqués apparaissent-elles ?
15. Comment être rapidement sûr que cette CRL est bien la bonne (émise par la bonne autorité de certification) ?
Argumenter la réponse
16. Vérifiez manuellement si le numéro de série du certificat du site *cyber.gouv.fr.pem* se trouve dans la CRL.
Décrire la méthode employée.

La révocation

Se rendre sur le site <https://revoked-rsa-dv.ssl.com/>

Essayez avec les 3 navigateurs. Qu'affichent-ils ?

Récupérez le certificat du site <https://revoked-rsa-dv.ssl.com/> et l'enregistrer dans le fichier *revoked-rsa-dv.ssl.com.pem*

17. De la même manière que tout à l'heure, vérifiez si le numéro de série du certificat se trouve dans la CRL de l'autorité de certification.
D'après vos observations, commentez.

Récupérez l'URL du répondeur OCSP du site <https://revoked-rsa-dv.ssl.com/>

Récupérez également le certificat de l'autorité de certification du site (seule l'autorité subordonnée est nécessaire, pas la racine). L'enregistrer dans un fichier séparé.

18. A l'aide de la commande OpenSSL **ocsp** vérifiez le statut du certificat en ligne de commande avec le protocole OCSP.
Indiquez la ligne de commande utilisée et commenter le résultat
(indice : vous aurez besoin du certificat du site, de l'autorité, de l'URL du répondeur OCSP de l'autorité. Utilisez également l'option *-no_nonce*)

Récupérez les certificats des sites <https://google.com> et <https://youtube.com>

19. Affichez leur numéro de série respectif.
Commentez le résultat. Expliquez cet état de fait

Se rendre sur <https://wrong.host.badssl.com/>

20. Qu'affiche le navigateur ?
Est-ce cohérent ? Argumenter malgré le fait que le certificat soit d'un type particulier.
(indice : <https://www.rfc-editor.org/rfc/rfc6125>)

Récupérer les 2 fichiers de certificats (isrg-1.crt et isrg-2.crt).

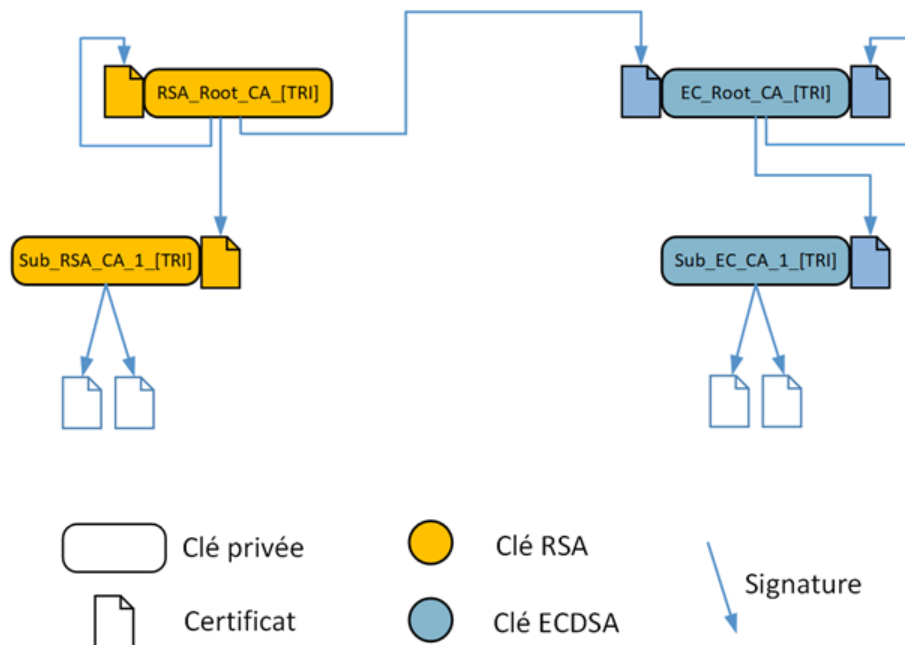
21. Les afficher, les examiner et les commenter

Partie 2

Créer les autorités de certifications suivantes. Le TP peut s'effectuer sur la VM Windows Server ou Linux (même si la plupart des tutoriels et cours sur internet sont sur une base Linux).

(Inspiré de la hiérarchie des CA de Let's Encrypt pré juin-2024

<https://letsencrypt.org/certificates/#old-content>)



Dans le nom des certificats d'autorité [TRI] désigne le trigramme des trinômes du TP (1^{ère} lettre de chaque nom)

Remarques :

- On sera attentif aux extensions des certificats (en particulier **X509v3 Basic Constraints** et **X509v3 Key Usage**)
- Limiter la distance aux entités finales : on mettra en place des contraintes de longueur sur le parcours de certification.
 - L'autorité racine RSA_Root_CA_[TRI] ne peut signer que 2 niveaux de d'autorités de certification subordonnée.
 - L'autorité racine EC_Root_CA ne peut signer qu'un niveau de d'autorité de certification subordonnée.
 - Les autorités subordonnées Sub_* ne pourront signer que des certificats d'entités finales.
- Les AC racines doivent être valables 15 ans
- Les AC subordonnées doivent être valables 10 ans
- Les certificats finaux doivent être valables 1 an
- Choisir les algorithmes de signature / tailles de clés et les algorithmes de hachage en fonction des recommandations de l'ANSSI en vigueur (aide : il existe un guide concernant les règles et

- recommandations concernant le choix et le dimensionnement de mécanismes cryptographiques) et prenant soin aux dates de validité demandées
- La certification croisée entre les 2 AC racines n'est que dans un sens

Qu'induit précisément le fait que la certification croisée ne soit que dans un sens ? Donnez un exemple concret.

Compléter le schéma si l'on souhaitait que la certification croisée soit dans les 2 sens au niveau des 2 AC racines.

Mise en place des autorités

1. Décrire les méthodes employées et comment sont appliquées les exigences ci-dessous.

Note : prévoir un fichier de configuration OpenSSL par AC avec des arborescences dédiées. De même les fichiers de database des AC (*.db) doivent être créés (fichier vide mais existant) avant d'utiliser les commandes OpenSSL (notamment **ca**).

Révocation par CRL

2. Emettre un certificat d'entité finale par l'autorité Sub_RSA_CA_1_[TRI]
3. A l'aide d'OpenSSL, valider le certificat face à la chaîne d'autorité de certificats
4. Révoquer ce même certificat et émettre un fichier de CRL puis afficher la CRL

Signature et chiffrement S/MIME

5. Emettre un certificat d'entité finale par l'autorité Sub_RSA_CA_1_[TRI] ayant comme finalité d'usage de clé : la signature et le chiffrement de mail S/MIME (note : les bonnes pratiques devraient faire générer 2 certificats : un pour la signature et un pour le chiffrement mais la méthode est simplifiée dans le TP).
Prendre un soin particulier aux usages du certificat
Décrire la méthodologie
6. Echanger avec un autre groupe le certificat puis envoyer à ce même groupe un message au format S/MIME **signé et chiffré** à l'aide d'OpenSSL
7. Réceptionner le fichier signé/chiffré de l'autre groupe et le déchiffrer et vérifier la signature.
Décrire la méthodologie
(il n'y a pas besoin de mettre en place des serveurs de mail. Echanger simplement les fichiers par clé USB, transfert SSH, etc.)

Révocation par OCSP

8. Emettre un certificat d'entité finale par l'autorité Sub_RSA_CA_1_[TRI] contenant
 - a. l'URL de récupération de la CRL (inventer un serveur/nom de domaine pour l'URI)
 - b. l'URL du répondeur OCSP (là encore inventer un serveur/nom de domaine. Le répondeur OCSP doit être au format URI)

Afficher le certificat

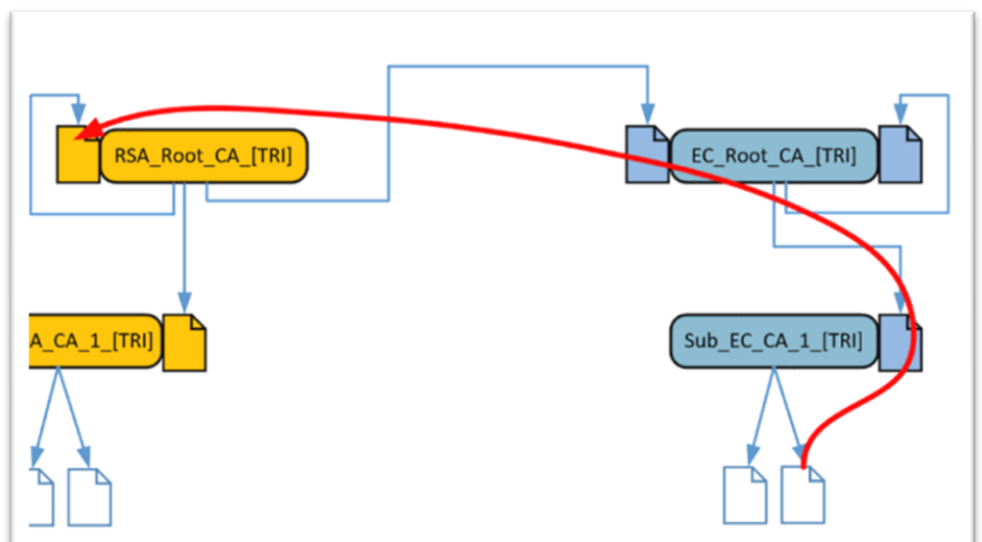
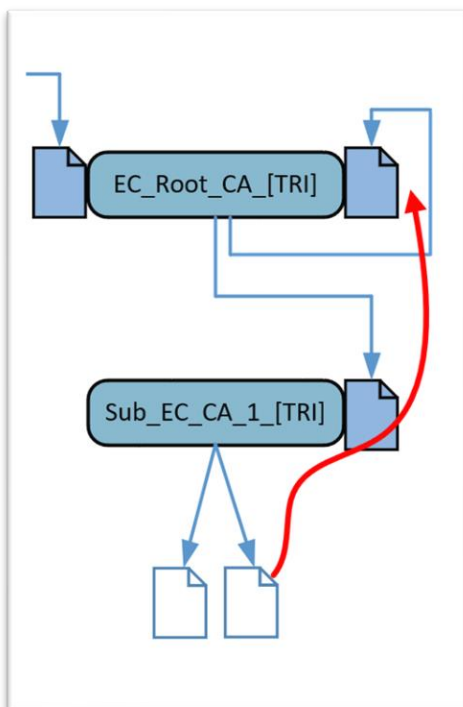
9. Mettre en place un répondeur OCSP à l'aide d'OpenSSL pour l'autorité Sub_RSA_CA_1_[TRI]
10. Vérifier le certificat créé précédemment avec le protocole OCSP. Le statut devrait être OK.

Afficher le résultat de la commande

11. Révoquer le certificat
 12. Vérifier à nouveau le certificat avec le protocole OCSP. Le statut devrait être KO.
- Afficher le résultat de la commande

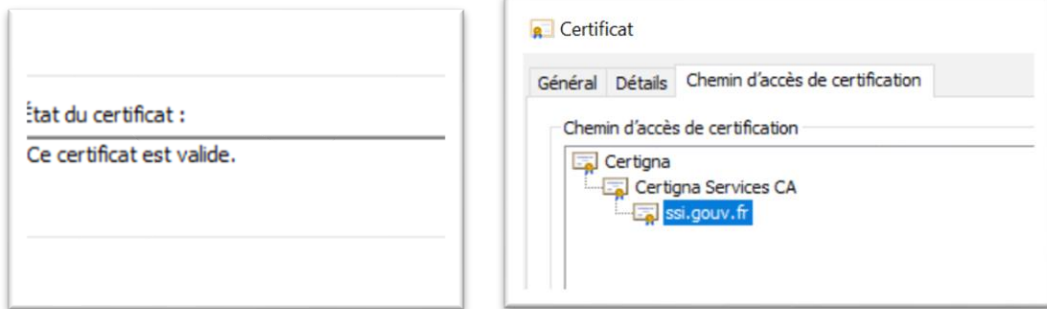
Certification croisée

13. Emettre un certificat final par l'autorité Sub_EC_CA_1_[TRI]
14. A l'aide d'OpenSSL vérifier les 2 chaînes de certification possibles (jusqu'au certificat auto-signé EC_Root_CA_[TRI] ou en remontant à la chaîne parallèle jusqu'au certificat RSA_Root_CA_[TRI])



Magasin de certificats

15. Selon la VM utilisée, exporter le certificat créé précédemment à l'aide de l'autorité Sub_EC_CA_1_[TRI] sous la machine virtuelle Windows.
Faire en sorte que le certificat soit affiché comme valide par Windows avec sa chaîne de certification. Par ex :



Proposition de méthodologie de construction de l'architecture des CA

=====

A faire pour chaque AC

=====

Root CA RSA

1. Création du fichier de configuration OpenSSL
2. Création d'une arborescence de répertoires dédiés
3. Création d'une requête de certificat (qui génère le bi-clé RSA)
4. Auto-signature de la requête

Sub CA RSA

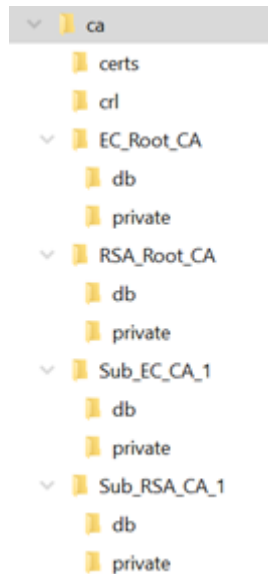
1. Création du fichier de configuration OpenSSL
2. Création d'une arborescence de répertoires dédiés
3. Création d'une requête de certificat (qui génère le bi-clé RSA)
4. Signature de la requête par l'AC Root

Root CA ECDSA

1. Création du fichier de configuration OpenSSL (ne supporte pas ECDSA)
2. Création d'une arborescence de répertoires dédiés
3. Création d'une clé ECDSA
4. Création d'une requête de certificat en utilisant la clé générée auparavant
5. Auto-signature de la requête

Sub CA ECDSA

1. Création du fichier de configuration OpenSSL (ne supporte pas ECDSA)
2. Création d'une arborescence de répertoires dédiés
3. Création d'une clé ECDSA
4. Création d'une requête de certificat en utilisant la clé générée auparavant
5. Signature de la requête par l'AC Root



Dans les dossiers « private » on stocke la clé privée de l'AC en question

Nom

 RSA_Root_CA.key

Dans les dossiers « db », on stocke les fichiers de database d'OpenSSL

 RSA_Root_CA.crt.srl
 RSA_Root_CA.crt.srl.old
 RSA_Root_CA.db
 RSA_Root_CA.db.attr
 RSA_Root_CA.db.attr.old
 RSA_Root_CA.db.old
 RSA_Root_CA.srl

Note : 4 des fichiers suivants sont à créer avant d'utiliser la commande **openssl ca**

RSA_Root_CA.crt.srl => le remplir avec « 01 »

RSA_Root_CA.db => doit être vide de base

RSA_Root_CA.db.attr => doit être vide de base

RSA_Root_CA.srl => le remplir avec « 01 »

Exemples de fichiers de configuration OpenSSL (à **modifier** !)

Pour une AC racine <https://bitbucket.org/stefanholek/pki-example-1/src/master/etc/root-ca.conf>

Pour une AC déléguée : <https://bitbucket.org/stefanholek/pki-example-1/src/master/etc/signing-ca.conf>

Partie 3

Rappels : une autorité de certification racine de confiance doit s'appuyer sur des mécanismes cryptographiques éprouvés, en particulier concernant la génération et le stockage des clés. Dans la plupart des cas un matériel dédié est utilisé (carte à puce ou HSM).

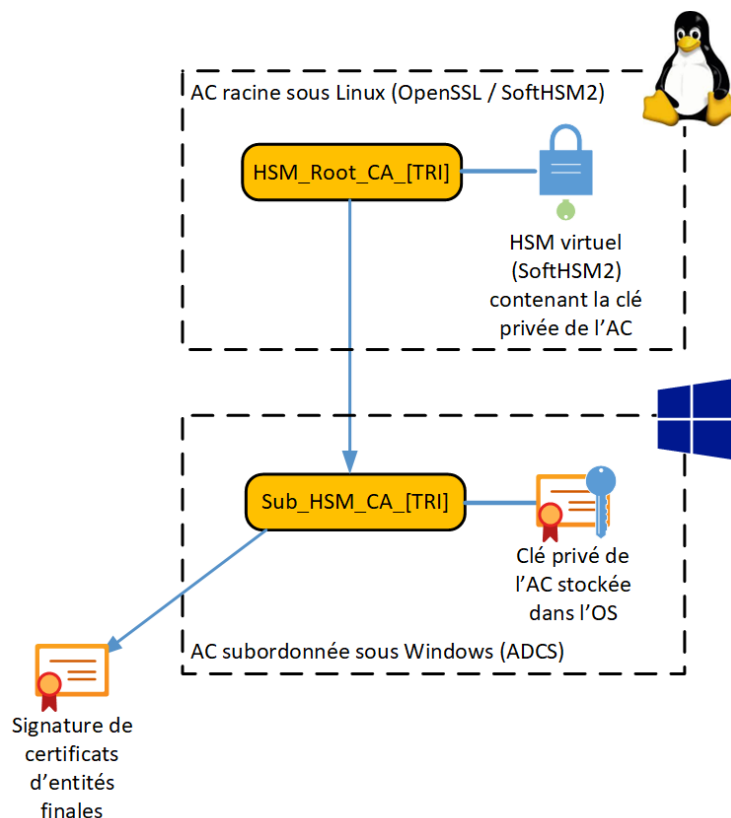
Afin de communiquer avec de tel matériel, le protocole PKCS#11 est le plus souvent utilisé.

Pour des raisons pratiques et financières, nous n'utiliserons pas de HSM ou de carte à puce physique dans ce TP. En revanche il est possible d'utiliser un HSM virtuel à des fins d'apprentissage, utilisable de la même manière qu'un HSM physique.

Pour cela nous utiliserons **SoftHSM2** qui est une librairie PKCS#11 émulant un HSM software.

Le but de ce TP est double :

- 1) Créer une autorité racine sous Linux à base d'OpenSSL s'appuyant sur **SoftHSM2** pour toutes les opérations cryptographiques (génération de clés, signature, etc.) impliquant la clé privée de l'AC racine.
- 2) Créer une autorité subordonnée sous Microsoft AD CS signée par l'autorité racine (sous Linux) à l'aide la VM Windows Server à votre disposition. Note : l'autorité de certification AD CS sera de type **Standalone** (non intégrée à un schéma Active Directory) à la différence du type *Enterprise CA*.



Autorité racine

L'autorité racine sera donc à implémenter sous base Linux avec la boîte à outils OpenSSL en conjonction avec SofHSM2.

Aucun fichier de clé privée ne devra être généré par l'OS (i.e. pas de fichier *.pem ou *.key). La clé privée devra être générée dans le HSM virtuel directement et toutes les opérations ayant besoin de cette clé devront se faire également via des appels PKCS#11 pour simuler l'utilisation d'un périphérique matériel de type HSM.

Note : en plus du logiciel **SoftHSM2** vous aurez également besoin de la suite **OpenSC** et plus particulièrement de la commande **pkcs11-tool**. Le package **libengine-pkcs11-openssl** sera également nécessaire.

De plus, l'utilisation de **SoftHSM2** nécessite au préalable l'initialisation d'un token (instanciation d'un module cryptographique pour stocker des clés) à l'aide des binaires **SoftHSM2** (**softhsm2-util**) ou via une interface PKCS#11.

Le but est de :

- Créer une AC racine autosignée avec une clé RSA 4096 bits stockée dans le HSM virtuel
- Créer un certificat racine d'AC valable 10 ans
- Créer une ARL valable 1 mois

Autorité subordonnée

L'autorité secondaire sera à créer sous machine virtuelle Windows Server 2019 à l'aide du composant ADCS (Active Directory Certificate Services) en mode Standalone.

Idéalement toutes les étapes de création de cette autorité secondaire seront à scripter en PowerShell. Une solution « graphique » via la GUI sera également acceptée mais l'effort de scripting sera apprécié et noté en conséquence.

Le but est de :

- Installer et configurer le rôle ADCS en mode Standalone
- Créer une autorité subordonnée (et donc créer une requête de certificat – CSR - au format PKCS#10) – clé RSA 4096 bits, valable 5 ans.
- Faire signer la requête de certificat de l'AC fille par l'AC racine OpenSSL
- Finir la configuration de l'autorité subordonnée
- Créer une requête de certificat pour **un site Web TLS** à l'aide de l'outil Windows **certreq** (ou en PowerShell) et la faire signer par l'autorité subordonnée à l'aide de l'outil **certutil** (ou en PowerShell)